

PARTNER INTEGRATION GUIDE

IACMS Unified API & Webhook Service

Integration reference for partner agencies and external systems

1. Introduction

This document specifies how partner agencies and external systems integrate with the IACMS platform. It covers two components of the platform's integration layer: the Unified GraphQL API, used for reading and writing case data on request, and the Webhook Service, used to receive real-time event notifications without polling.

Both components are designed around a single principle: partner integrations should be simple to build against, precise in the data they exchange, and structurally incapable of accessing data belonging to another tenant.

1.1 What's Covered

- The Unified GraphQL API — a single endpoint for querying and modifying case data.
- The Webhook Service — subscription management and real-time event delivery.
- Authentication, permission scopes, rate limits, and error handling.
- Payload verification and delivery behavior for webhooks.

2. Quickstart

The fastest way to confirm your credentials work is a single metrics query:

1. **Generate an API key** from the agency settings portal. The raw key is shown only once — store it securely.
2. **Send a POST request** to the GraphQL endpoint with your key in the `X-API-Key` header (see Section 4).
3. **Confirm you receive a 200 response** with no errors array.

```
curl -X POST https://api.iacms.gov/api/v1/graphql \ -H "Content-Type: application/json" \ -H "X-API-Key: iacms_live_..." \ -d '{"query": "query { metrics { totalCases } }"}'
```

3. Environments & Versioning

Environment	Base URL	Notes
Production	<code>https://api.iacms.gov</code>	API keys are prefixed <code>iacms_live_</code> .

The API is versioned by URL path — the current version is `/api/v1/`. Any future changes to the contract will be communicated in advance through your IACMS technical contact.

4. Authentication

4.1 Obtaining an API Key

API keys are issued from the agency settings portal. Each key is shown in full exactly once, at creation — IACMS stores only a SHA-256 hash of the key afterward and cannot redisplay it. If a key is lost, revoke it and generate a new one.

```
X-API-Key: iacms_live_...
```

An API key is issued per tenant. The platform resolves the key to its owning tenant automatically; clients are not required to, and cannot, specify a tenant identifier on any request.

4.2 Permission Scopes

Every API key is granted one or more scopes, which gate access to individual queries and mutations. A request that touches a field or operation outside the key's granted scopes is rejected (see Section 8).

Category	Scopes
Read	<code>cases:read</code> , <code>workflows:read</code> , <code>workflowSteps:read</code> , <code>referrals:read</code> , <code>assignments:read</code> , <code>auditLogs:read</code> , <code>departments:read</code> , <code>metrics:read</code>
Write	<code>cases:create</code> , <code>cases:update</code> , <code>referrals:create</code> , <code>users:create</code> , <code>users:update</code> , <code>users:deactivate</code>

4.3 Transport Security

TLS 1.3 is required for all requests. Network-level access controls, including IP allow-listing, can be configured for your integration — contact your IACMS technical contact to set this up.

5. Core Concepts

Queries — used to fetch data, such as retrieving a list of cases or looking up a single record.

Mutations — used to change data, such as creating a case or advancing it to its next workflow step.

Both operation types are submitted as a single HTTP POST to the endpoint below.

Endpoint	POST <code>https://api.iacms.gov/api/v1/graphql</code>
Format	GraphQL over HTTPS

6. Querying Data

6.1 Available Entities

Query	Returns
<code>cases</code>	Case records
<code>workflows</code>	Workflow definitions
<code>workflowSteps</code>	Steps within a workflow
<code>referrals</code>	Inter-agency referrals
<code>assignments</code>	Case assignments
<code>auditLogs</code>	Audit trail entries
<code>departments</code>	Agency departments
<code>metrics</code>	Aggregate platform metrics

Example — Fetching Urgent Cases

```
query GetUrgentCases { cases( filter: { priority: { eq: "critical" }, status: { eq: "open" } } pagination: { limit: 10, offset: 0 } ) { data { caseNumber title status assignee { email } currentStep { name } } pagination { total hasMore } } }
```

6.2 Filter Operators

Field Type	Operators	Purpose
String	eq, in	Exact match / match against a list of values
Text (search)	contains	Substring match
Date / Time	gte, lte	Range bounds — on or after / on or before

Filtering and sorting are validated against an explicit, per-entity allowlist of fields. A request that filters or sorts on a field outside that allowlist is rejected with an INVALID_QUERY error (Section 8).

6.3 Pagination

Parameter	Type	Default	Maximum
<code>limit</code>	Integer	<code>20</code>	<code>100</code>
<code>offset</code>	Integer	<code>0</code>	—

Pagination is offset-based. Every paginated query returns a pagination object alongside the data, including total and hasMore, as shown in the example above.

6.4 Query Complexity Limits

Every query is scored for cost before execution and rejected if it exceeds the platform's limits. This keeps the platform fast and available for every partner, and requires no special handling for well-formed, reasonably shallow queries.

Limit	Value
Maximum depth	<code>10 levels</code>
Maximum total cost	<code>500</code>

Cost Factor	Weight
Base scalar field	<code>1</code>
Relation field	<code>10</code>
Nested relation (per additional level)	<code>50</code>
Text search (contains)	<code>20</code>
in filter	<code>5 per list item</code>

Queries that request many nested relations, or combine several text-search filters, accumulate cost quickly — keep selection sets focused on the fields you actually need.

7. Modifying Data (Mutations)

Mutation	Purpose
<code>createCase</code>	Open a new case
<code>updateCase</code>	Edit an existing case
<code>closeCase</code>	Close a case
<code>executeTransition</code>	Advance a case to its next workflow step
<code>createReferral</code>	Send a referral to another agency
<code>inviteUser</code> / <code>updateUser</code> / <code>deactivateUser</code>	Manage the organization's users

Example — Creating a Case

```
mutation CreateEmergencyCase { createCase(input: { workflowKey: "child-protection" title: "Emergency Referral" type: "child_protection" priority: critical }) { caseId caseNumber status } }
```

8. Error Handling

Errors can originate from two layers: the authentication middleware, which runs before your request reaches the GraphQL engine, and the GraphQL engine itself, once a request has been authenticated.

8.1 Authentication Errors

If your API key is missing, invalid, or expired, the request is rejected before execution with a plain JSON error envelope:

```
{ "success": false, "error": { "code": "INVALID_API_KEY", "message": "The provided API key is invalid or has expired.", "details": { "resolution": "Generate a new API key in the agency settings portal." } }, "meta": { "requestId": "err_91823bc0" } }
```

8.2 GraphQL Execution Errors

Once a request is authenticated, successful responses do not include an errors field. When a request cannot be completed, the response includes an errors array; inspect each error's extensions.code:

Code	HTTP Status	Meaning
UNAUTHORIZED	401	Missing, invalid, or expired API key
FORBIDDEN	403	Key lacks the required scope for this field
INVALID_QUERY	400	Malformed request, or an invalid/non-allowlisted field selection
BUSINESS_RULE_VIOLATION	422	The request is well-formed but violates a workflow rule
RATE_LIMITED	429	Quota exceeded — see Section 9
QUERY_TOO_DEEP	400	Exceeded the maximum query depth (10)
QUERY_TOO_COMPLEX	400	Exceeded the maximum query cost (500)

Clients should check for the errors field on every response, independent of the HTTP status code returned.

9. Rate Limits & Quotas

Limit	Value
General API requests	100 requests / 60 seconds, per key
Authentication-related requests	10 requests / window

Exceeding a limit returns an HTTP 429 with extensions.code RATE_LIMITED. Respect the Retry-After header before sending further requests.

10. Code Examples

cURL

```
curl -X POST https://api.iacms.gov/api/v1/graphql \ -H "Content-Type: application/json" \ -H "X-API-Key: iacms_live_..." \ -d '{"query": "query { metrics { totalCases } }"}'
```

Python (requests)

```
import requests response = requests.post( "https://api.iacms.gov/api/v1/graphql", headers={"X-API-Key": "iacms_live_..."}, json={"query": "query { cases(pagination: {limit: 5}) { data { caseNumber } } }"} )
```

11. Webhook Service

The Webhook Service allows a tenant to subscribe to real-time platform events and receive them as they occur, removing the need to poll the API for changes.

11.1 Authentication for Webhook Management

Human session required

Webhook subscriptions can only be created, modified, or deleted by an authenticated user through the IACMS dashboard (a human session). API keys cannot be used to manage webhook subscriptions. This is a deliberate security control.

11.2 Managing Subscriptions

Method	Endpoint	Description
POST	/api/v1/webhooks	Create a subscription. Returns the signing secret — see 11.4.
GET	/api/v1/webhooks	List all subscriptions for the tenant.
GET	/api/v1/webhooks/:id	Retrieve a single subscription.

11.3 Event Types

Category	Events
Cases	<code>case.created</code> , <code>case.status_changed</code> , <code>case.closed</code>
Referrals	<code>referral.created</code> , <code>referral.accepted</code> , <code>referral.rejected</code> , <code>referral.completed</code>
Assignments	<code>assignment.created</code> , <code>assignment.removed</code>

Use GET `/api/v1/webhooks/events` at any time for the authoritative, current list of supported events.

11.4 Creating a Subscription

The signing secret is shown once

The 32-byte webhook secret is returned only in the response to the POST `/api/v1/webhooks` request that creates the subscription. It cannot be retrieved again afterward. Store it securely at creation time — if it is lost, the subscription must be deleted and recreated to obtain a new secret.

Example Request

```
POST /api/v1/webhooks Content-Type: application/json (authenticated dashboard session) { "url": "https://partner.example.org/webhooks/iacms", "events": ["case.created", "referral.created"], "isActive": true }
```

Example Response

```
{ "id": "wh_8f2a1c...", "url": "https://partner.example.org/webhooks/iacms", "events": ["case.created", "referral.created"], "isActive": true, "secret": "a1b2c3d4...e9f0" // shown once only }
```

Sample Payload — case.created

```
{ "event": "case.created", "tenantId": "550e8400-e29b-41d4-a716-446655440000", "requestId": "req_8923bc71a", "timestamp": "2026-08-17T11:00:00.000Z", "data": { "caseId": "e1829a0b-1122-4372-a567-0e02b2c3d999", "caseNumber": "DCS01-2026-0501", "status": "open", "currentStep": "Intake Desk", "createdAt": "2026-08-17T11:00:00.000Z" } }
```

Every event shares this same envelope — event, tenantId, requestId, timestamp, and data — with the data object shaped to the resource that changed.

11.5 Verifying Webhook Payloads

Every webhook delivery includes an X-IACMS-Signature-256 header. This header contains an HMAC-SHA256 hash of the raw JSON request body, computed using the secret returned at subscription creation, in the format:

```
X-IACMS-Signature-256: sha256=<hex digest>
```

Clients must verify this signature before trusting or acting on a webhook payload. A payload should be discarded if the computed signature does not match the header.

Verification Example — Node.js

```
const crypto = require('crypto'); function verifySignature(rawBody, signatureHeader, secret) { const expected = 'sha256=' + crypto.createHmac('sha256', secret).update(rawBody).digest('hex'); return crypto.timingSafeEqual( Buffer.from(expected), Buffer.from(signatureHeader) ); }
```

Verification Example — Python

```
import hmac, hashlib def verify_signature(raw_body: bytes, signature_header: str, secret: str) -> bool: expected = 'sha256=' + hmac.new( secret.encode(), raw_body, hashlib.sha256 ).hexdigest() return hmac.compare_digest(expected, signature_header)
```

11.6 Delivery Behavior

Property	Value
Delivery attempts	Up to 4 (1 initial + 3 retries)
Backoff schedule	1s, 2s, 4s between retries (exponential)
Per-attempt timeout	15,000 ms

Deliveries are signed, timed out, and retried automatically as shown above, so a brief interruption on your endpoint is absorbed without any action on your part. We recommend keeping your receiving endpoint highly available and responding quickly (2xx) so events are acknowledged well within the delivery timeout.

11.7 Testing Your Integration

We recommend pointing a new subscription at a public request-inspection tool (such as webhook.site) during initial development to inspect payloads and headers, and using a tunneling tool (such as [ngrok](https://ngrok.io)) to test end-to-end against a local development server before going live.

11.8 Best Practices

- Store the signing secret in a secrets manager, never in source control or plaintext configuration.
- Always verify the signature against the raw request body, before any JSON parsing or transformation.
- Respond to webhook deliveries quickly (2xx) and process the payload asynchronously to avoid delivery timeouts.
- Design event handling to be idempotent — the same event may occasionally be delivered more than once.
- Use `GET /api/v1/webhooks/events` to confirm the current, authoritative list of supported events rather than hard-coding it.

12. SDKs & Client Libraries

No SDK installation is required. The API surface is a single endpoint, so integrating via your language's standard HTTP client — as shown in Section 10 — is all that's needed.

13. Support

For questions regarding API scopes, webhook delivery, event coverage, or integration issues, please